

Generate-and-Test Trace Features

Status: independent negative/redesign proposal. The current experiment is not positive evidence for generate-and-test; it is evidence that the utility rule and testbed must be redesigned before promotion.

Abstract

Continual agents need representations that can change while learning online. This proposal studies a minimal version of that problem: a prediction agent has a small budget of temporal trace features, the reward delay changes during the stream, and the agent must decide which traces to keep or replace. The intended hypothesis was that utility-based generate-and-test replacement would recover useful timescales faster than fixed trace banks or random replacement. The pilot does not support that hypothesis. Generate-and-test often moves active trace timescales closer to the new delay, but it does not reduce prediction error relative to fixed or random baselines. The result is therefore a negative redesign result: the experiment identifies a real Core RL question, but the current utility proxy is not yet aligned with downstream prediction improvement.

Study Claim And Evidence Level

This is a standalone representation-learning proposal. Its claim is deliberately limited:

> In the current delay-switch trace-prediction stream, the implemented generate-and-test utility rule changes the feature set but does not improve downstream prediction error.

The evidence level is negative/redesign. The result should not be described as a successful feature-learning result, because the primary behavioral metric does not improve and the oracle trace bank is not a validated upper bound.

Research Motivation/Question/Method

The Alberta Plan emphasizes ordinary experience, continual learning, limited computation, value functions, learned models, planning, and feature finding. A long-lived agent cannot keep every possible feature. Reward delays, sensor statistics, and task-relevant history lengths can change while the agent continues to act and predict. A fixed representation either wastes capacity on stale features or requires manual retuning by the experimenter.

Generate-and-test is attractive because it gives representation maintenance an online form: generate candidate features, test their utility from the data stream, and replace low-utility features. The hard part is not generating plausible features. The hard part is deciding whether a feature improves the agent's prediction or control objective. A trace with a timescale close to the reward delay may still be redundant, badly scaled, or poorly coupled to the current update rule. This proposal tests that mismatch directly.

Focused RL Question

The focused RL question is:

> Under a fixed trace-feature budget, can utility-based generate-and-test replacement preserve or recover useful temporal traces after the reward delay changes?

The operational hypothesis was:

> A useful generate-and-test rule should improve post-change recovery and late prediction error compared with fixed trace banks and random replacement.

The pilot falsifies this operational hypothesis for the implemented utility rule. It does not falsify generate-and-test in general.

RL Setting

The environment is a streaming trace-conditioning prediction problem:

- - a cue appears in the observation stream;
- - reward appears after a delay;
- - halfway through the stream, the delay changes from 10 to 20;
- - the learner sees the stream once, with no replay buffer and no offline training phase;
- - the feature budget is small, so the learner cannot include every plausible trace timescale.

This setting isolates a continual representation question. The environment is intentionally smaller than a control task so that feature replacement events, trace timescales, and prediction errors can be audited directly.

Method

The compared feature strategies are:

- - fixed tight trace bank;
- - oracle-style trace bank;
- - random replacement;
- - generate-and-test replacement.

All methods use the same streaming prediction problem and the same limited feature budget. The generate-and-test condition estimates feature utility online and replaces low-utility traces with newly sampled traces. The random replacement condition tests whether churn alone helps. The oracle-style bank is intended as a diagnostic matched-feature condition, not as a proven upper bound; its weak performance in the pilot is one reason this proposal remains in redesign.

The study separates two forms of evidence:

- - structural evidence: whether active trace timescales move closer to the current delay;
- - behavioral evidence: whether prediction error and recovery improve.

Only the second form can support a positive representation-learning claim.

Experimental Design

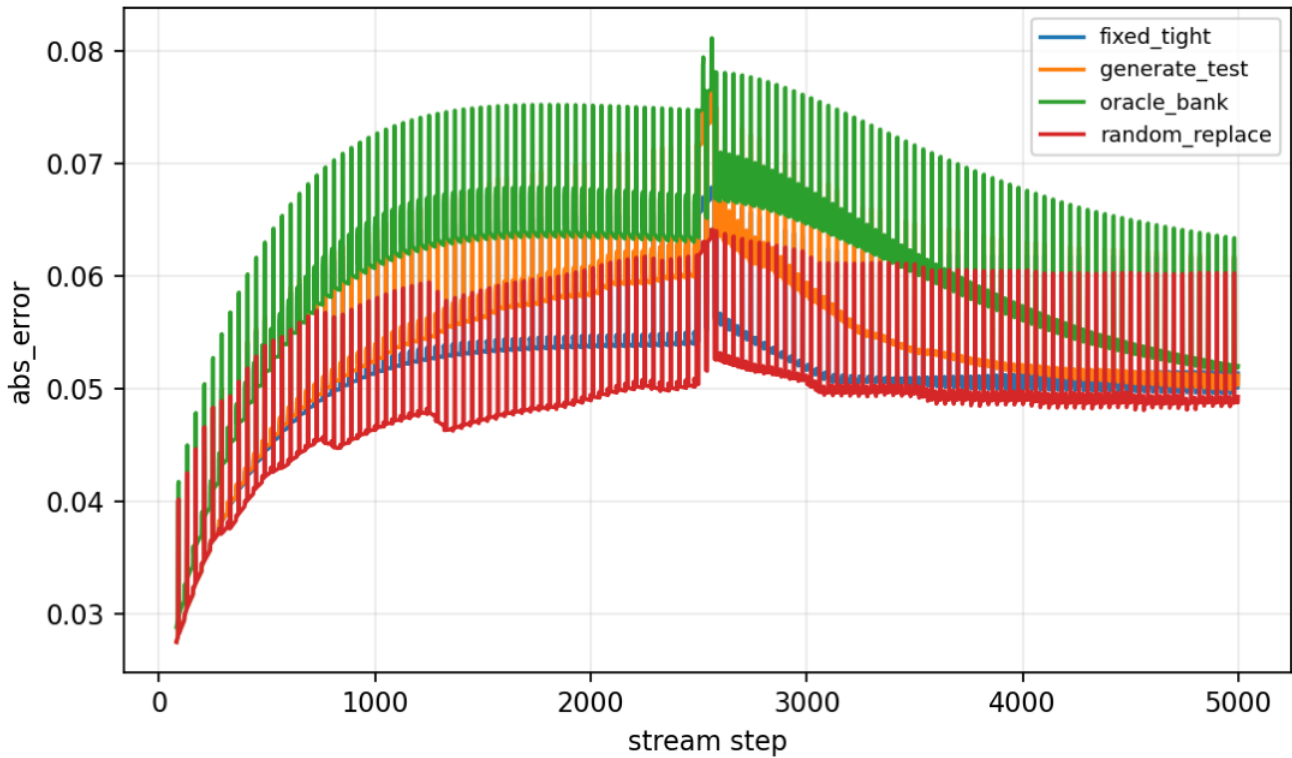
Current main pilot:

- - Feature budget: 4 traces.
- - Delay switch: halfway through the stream.
- - Seeds: 0-4.
- - Steps: 5000.
- - Result path:
experiments/alberta_core_rl/results/generate_test_features/20260708T154941Z_main.

Primary metrics:

- - absolute prediction error;
- - late post-change absolute prediction error;
- - recovery-window summaries around the delay switch;
- - distance from active trace timescales to the current target delay;
- - replacement count and feature survival.

Primary figure:



Delay-shift prediction error by feature strategy.

Results

Generate-and-test often keeps active trace timescales closer to the target delay than fixed or random traces. That diagnostic shows that replacement is doing something interpretable. However, the improvement is structural rather than behavioral. It does not produce lower prediction error.

Late post-change absolute error is approximately:

Strategy	Late post-change absolute error
Random replacement	0.0496
Fixed tight traces	0.0512
Generate-and-test	0.0518
Current oracle-style bank	0.0545

The oracle-style result is a warning sign. If the supposedly matched trace bank does not clearly beat generic baselines, the environment is not yet a clean test of feature selection. The pilot therefore exposes a design failure rather than a successful generate-and-test mechanism.

Analysis

The negative result is informative because the feature diagnostics and the

prediction metric disagree. The likely mechanisms are:

- 1. Utility is misaligned with the prediction objective. The implemented score can prefer traces that look active or well timed without estimating their marginal reduction in prediction error.
- 2. The feature bank may be redundant. Several trace timescales can support similar predictions, so moving closer to the nominal delay may not change the linear predictor enough to improve error.
- 3. The stream may be underpowered. A single delay switch may not create enough sustained pressure for utility replacement to matter.
- 4. The oracle is not validated. If the oracle-style bank is not a reliable winner, the testbed cannot distinguish a weak utility rule from an environment where trace choice barely matters.
- 5. Replacement may disrupt learning. Replacing features changes the input basis while weights are still adapting, so a feature that is structurally better can temporarily hurt prediction.

These mechanisms keep the interpretation narrow. The study says that the current utility rule is insufficient for this stream, not that feature generation is unimportant.

Threats To Validity

The pilot uses one delay switch and a small number of seeds. A stronger study should test repeated switches, multiple delay pairs, and feature-budget sweeps.

The oracle-style baseline is not yet a true upper bound. Before any positive claim, the testbed must include a known trace bank that reliably improves prediction error.

The current utility rule is compact and should not be treated as a representative implementation of all generate-and-test methods.

The task is prediction-only. A downstream control setting might value traces differently, but that would require a new experiment rather than a reinterpretation of this pilot.

Next Experiments

This proposal should move forward only as a redesign:

- 1. Build a validation stream where a hand-specified trace bank reliably beats generic trace banks on prediction error.
- 2. Add repeated nonstationary delay changes so that stale features create sustained pressure.
- 3. Replace the current utility proxy with a measure tied to downstream loss, such as estimated contribution to TD-error reduction or recovery speed.
- 4. Run a budget sweep to identify when trace capacity is actually scarce.
- 5. Keep random replacement and fixed banks as baselines, and report both feature diagnostics and prediction metrics.

The promotion criterion is simple: generate-and-test must improve recovery or late prediction error under the same online, limited-budget conditions.

Alberta Plan Connection

The proposal connects to Alberta Plan feature finding, ordinary experience, continual adaptation, and limited computation. It also follows the idea that

learned components should be tested by utility rather than by plausibility. The experiment remains small and linear so that the mechanism can be inspected before moving to more complex prediction or control settings.

Local references:

- - resources/alberta_plan_related/learning_agent_state_online_2112.15236.pdf
- - resources/alberta_plan_related/alberta_plan_2208.11173.pdf

Reviewer Critique

Reviewer angle	Critique	Revision in this report	Remaining risk
Representation learning	Plausible feature dynamics are not evidence of useful representation learning.	The report separates structural diagnostics from downstream prediction error.	Needs utility tied to loss reduction.
Experimental design	The oracle-style bank is not a clear upper bound.	Evidence level is negative/redesign rather than positive.	Requires a validation stream where known traces win.
Core RL scope	The proposal must ask a focused RL question, not only rank methods.	The question is limited-budget online feature adaptation after a delay change.	Needs repeated nonstationarity to become stronger.
Strict interpretation	Do not overstate structural feature movement as positive evidence.	The report is marked independent and negative.	Future writing must preserve this evidence level.

Proposal Template Answers

Focused RL question: Under a limited feature budget, can generate-and-test replacement discover temporal trace features that improve nonstationary delay prediction?

Setting/testbed: A streaming trace-conditioning prediction task with a reward-delay switch from 10 to 20.

Implemented comparison: fixed tight traces, oracle-style traces, random replacement, and utility-based generate-and-test replacement.

Observation or metric: absolute prediction error, recovery after the delay switch, active trace timescale, replacement behavior, and feature survival.

Compute and fallback: CPU-scale, seeds 0-4, 5000 steps. The current fallback is a negative redesign result, not a positive feature-learning claim.

Conclusion

This is an independent proposal about limited-capacity online representation adaptation. Its contribution is the diagnosis: the current utility mechanism produces interpretable trace movement without downstream error improvement. The correct next step is not to promote the current generate-and-test rule, but to redesign the utility measure and validation stream so that a positive result would require improved recovery or lower late prediction error under the same online feature budget.

Reproduction

```
cd /mnt/shared-storage-user/yupeng/Core-RL
```

```
PYTHONNOUSERSITE=1 MPLCONFIGDIR=/mnt/shared-storage-user/yupeng/Core-RL/.mplconfig \  
/data/yupeng/conda_envs/core-rl/bin/python experiments/alberta_core_rl/scripts/run_experiment.py \  
--config experiments/alberta_core_rl/configs/generate_test_features/config_main.json
```